

What are the advantages of High-Throughput JPEG 2000 (HTJ2K) in OpenEXR for visual effects (VFX) compared to existing OpenEXR algorithms (e.g., ZIP, PIZ, DWAA, DWAB)?

A Comprehensive Study Comparing OpenEXR Compression Methods

Mats Valgaeren

Graduation work 2025-2026



Contents

Abstract & Keywords	3
Preface	5
List of figures	6
List of Tables	7
Introduction	8
1 Literature study / Theoretical framework	9
1.1 Compression Fundamentals	9
1.1.1 Data Compression	9
1.1.2 Decompression Details	10
1.1.3 Compression Details	10
1.1.4 Threading	10
1.2 OpenEXR Foundation	11
1.2.1 OpenEXR Creation	11
1.2.2 OpenEXR Compression Methods	12
1.2.3 Scanline vs. Tiled Storage	13
1.2.4 Arbitrary Channels and Multi-Layered files	14
1.2.5 Deep Compositing	15
1.2.6 Color Space & Linear Workflow	16
1.3 Raster Image Compression Background	18
1.3.1 JPEG History	18
1.4 HTJ2K Foundation	19
1.4.1 HTJ2K Background	19
1.4.2 Region of Interest	20
1.4.3 Partial Decoding	20

1.5	Current Research State	21
1.5.1	HTJ2K integration into OpenEXR	21
1.5.2	Current Integration	22
1.6	Research Gaps & Justification	23
2	Research	24
2.1	Experimental Setup	24
2.1.1	Dataset Characteristics	25
2.2	Benchmarking Environment	27
2.2.1	Hardware	27
2.2.2	Software Stack	27
2.2.3	Benchmark Pipeline	27
2.2.4	Nuke Integration Tests	30
2.3	Benchmark Results	30
2.3.1	Image Content Analysis	30
2.3.2	Codec Performance Ranking	32
2.3.3	Scanline versus Tiled Layout	33
2.3.4	Region-of-Interest Decompression	34
2.3.5	Nuke Timeline Validation	36
2.3.6	Summary and Key Findings	37
3	Case Study	38
3.1	Survey Design	38
3.1.1	Sampling Strategy	38
3.2	Respondent Profile and Sampling Characteristics	39
3.2.1	Respondents	39
3.2.2	Survey Characteristics and Implications	39
3.2.3	Software Tools by Career Stage	40
3.2.4	OpenEXR Format Knowledge	41
3.3	Compression Usage Results	42

3.3.1	Images Preference	42
3.3.2	File Size Priorities	44
3.3.3	Most Used OpenEXR Compression Methods	45
3.4	Linear vs. Progressive Decompression	46
	Discussion	48
	Conclusion	48
	Future work	49
	Reflection	50
	Acknowledgments	51
	Bibliography	53
	Appendix	56

Abstract & Keywords

[English] Every day, terabytes of OpenEXR image sequences are processed by visual effect pipelines, where compression directly affects storage costs and artist productivity. This paper investigates the performance of High-Throughput JPEG 2000 (HTJ2K), against its competitors. The research combines benchmarks with insights gathered from industry professionals.

The open-source HTJ2K implementation used by OpenEXR, OpenJPH, achieves compression speed and ratio almost equivalent to ZIP and PIZ. Its progressive decoding and region-of-interest (ROI) capabilities are functional but not utilized in current compositing workflows due to DCC tool architecture. Proprietary implementations or enhancements in the current library make it a clear winner in raw performance.

These benchmarks represent the best current understanding of HTJ2K's potential; pipeline integration will depend on future DCC integration.

[Nederlands] Dagelijks worden terabytes aan OpenEXR-beeldsequenties verwerkt door visuele effect-pipelines, waarbij compressie direct van invloed is op de opslagkosten en de productiviteit artiesten. Dit artikel onderzoekt de prestaties van High-Throughput JPEG 2000 (HTJ2K) ten opzichte van concurrenten. Het onderzoek combineert benchmarks met inzichten van professionals uit de industrie.

De open-source HTJ2K-implementatie die door OpenEXR wordt gebruikt, OpenJPH, behaalt een compressiesnelheid en -ratio die bijna gelijkwaardig is aan ZIP en PIZ. De progressieve decodering en de regio-van-interesse (ROI)-functionaliteit zijn functioneel, maar worden momenteel niet benut in compositingworkflows vanwege de architectuur van DCC-tools. Propriëtaire implementaties of verbeteringen in de huidige bibliotheek maken het een duidelijke winnaar op het gebied van pure prestaties.

Deze benchmarks geven een zo goed mogelijk beeld van het potentieel van HTJ2K; integratie in pipelines zal afhangen van toekomstige DCC-integratie.

Keywords: OpenEXR, HTJ2K, lossless compression, visual effects pipeline, image sequence optimization, high-throughput JPEG 2000

Preface

As a future Pipeline TD, I try to get to the bottom of topics and to know more than is necessary. This helps me prepare for potential issues and develop more effective solutions.

Image file compression has always interested me, as I am using these extensively in my career, especially OpenEXR as it is the default in VFX. When HTJ2K was introduced earlier this year in OpenEXR, I knew I had to learn more. Its potential as a groundbreaking implementation motivated me to delve deeper, with the hope of clarifying some of the misconceptions online and to provide useful guidance to my peers.

In addition to the main research topic, I also wanted to explore several related areas that could complement this project. These included NeoVim, Zotero, pandas, $\text{\LaTeX}$, performing CPU and speed benchmarks, and visualizing these through graphs.

List of Figures

1.1	Lossless vs Lossy Image Compression [4]	9
1.2	Threading [13]	10
1.3	AOV Types [23]	14
1.4	Deep Compositing Visuals [24]	15
1.5	Scene-Referred vs. Display-Referred [31].	16
1.6	JPEG Algorithm [36].	18
1.7	JPEG vs HTJ2K [45]	19
1.8	Results File Size [59].	22
2.1	Scripts Flow Diagram	28
2.2	Decode speed versus file size across image groups	31
2.3	Codec performance across single-core and 16-core configurations	32
2.4	Scanline versus tiled layout performance	33
2.5	Scanline versus tiled layout decompression on different regions	34
2.6	Nuke playback performance for lossless compression methods	36
3.1	Participant Skill Level	39
3.2	Software and Render Usage	40
3.3	OpenEXR Knowledge	41
3.4	Most Used Image Types in VFX Workflows	42
3.5	Most Used Image Formats in VFX Workflows	43
3.6	Importance of Read Speed, Write Speed, and File Size	44
3.7	Most Used OpenEXR Compression	45
3.8	Simulated Linear vs. Progressive Decompression (“Charge” Open Movie)	46
3.9	Simulated Linear vs. Progressive Decompression (DOGWALK)	46
3.10	Simulated Linear vs. Progressive Decompression (Lynxsdesign)	46
3.11	Progressive Decompression Survey Ratings	47
3.12	Rating Progressive Decompression vs. Job Level	47

List of Tables

1.1	OpenEXR Compression Methods [22][18]	12
1.2	Results Encoding and Decoding speed [59]	22
2.1	Images used in benchmark test (Netflix)	25
2.2	Images used in benchmark test (Reboot)	26
2.3	Friedman Test Rankings: ROI Decode Performance by Compression Method	35

Introduction

The lossless compression methods we are using today in the VFX industry are over 20 years old [1]. This means that they are well implemented, but there is a lot of room for improvement as our processors get faster and more efficient. A newer compression method, High Throughput JPEG 2000 (HTJ2K), was added to the OpenEXR library as of September 2025 [1]. And although software integration is still ongoing, it is already showing interesting results. This research evaluates HTJ2K's practical usefulness and offers VFX industry recommendations on compression choices.

This research aims to:

- Evaluate the usefulness of HTJ2K over existing OpenEXR algorithms.
- Deliver data-driven guidance for balancing speed, size, and quality and choosing a compression method

To achieve the objectives outlined above, the following research question guides this study:

RQ: What are the advantages of High-Throughput JPEG 2000 (HTJ2K) in OpenEXR for visual effects (VFX) workflows compared to established algorithms such as ZIP, PIZ, DWAA, and DWAB?

To address this question, the following hypotheses were formulated:

- H₀:** High-Throughput JPEG 2000 offers no statistically significant performance advantages over existing OpenEXR compression algorithms in typical VFX workflows.
- H₁:** HTJ2K features such as *Region of Interest (ROI)* decoding and *progressive decompression* provide measurable workflow benefits for VFX production pipelines.
- H₂:** HTJ2K achieves superior compression ratios while maintaining comparable or better decoding performance, offering advantages for VFX studios with constrained storage capacity.

Chapter 1: Literature study / Theoretical framework

Every file that is saved on our computers takes up storage, which itself costs money. We can try to minimize used storage by having less data, which is not easy to do in a world where we store a massive amount of files like images, applications, and games that we also share with the world. A better alternative is to decrease the space a file takes up with compression.

1.1 Compression Fundamentals

1.1.1 Data Compression

Data compression is the process of taking data and then encoding, restructuring, or modifying it in order to reduce its size in order to lower the cost of storage and/or transmission [2].

This is very important for digital images, as these tend to be very big and uncompressed. Image compression algorithms may take advantage of visual perception and statistical properties of image data to achieve smaller file sizes without visual differences [3].



Figure 1.1: Lossless vs Lossy Image Compression [4]

Image compression can be lossless or lossy. Lossless compression is preferred for archival, medical imagery, technical drawing, and movie creation [5], while lossy compression is used more when bitrates are limited and quality loss is acceptable, like for social media, web uploads, and streaming [5].

1.1.2 Decompression Details

Compressing data results in much smaller file sizes, but we have to keep in mind that we also have to decompress the file to make it possible to run or view.

To start the decompression of a file, an application first retrieves the compressed data from the secondary storage (e.g., SSD/HDD). It then saves it into the application's memory (typically RAM (Random Access Memory)) [6]. A single- or multi-threaded decoder then processes the data into uncompressed bytes; all these bytes often get saved in the RAM for fast access and can then be used by the application. When decompressing an image, the saved bytes represent the RGBA pixel values of every pixel, which are processed by the CPU's (Central Processing Unit) [7] integrated GPU (Graphics Processing Unit) [8], or a separate dedicated GPU for viewing purposes [9].

1.1.3 Compression Details

When compressing a file, we go the opposite way; the bytes (pixels of the image) are first copied to RAM [6]. This uncompressed (image) data is sent to the processor for single- or multi-threaded compression (into scanline or tiled blocks), which is then written back to the secondary storage.

1.1.4 Threading

Threading parallelizes compression/decompression across CPU cores [10], saving time for larger files by splitting the workload. Multi-threading divides scanlines or blocks among these threads [11], scaling with core count and minimizing RAM [6] overhead. This process is leveraged by some image formats to speed up processing time [12].

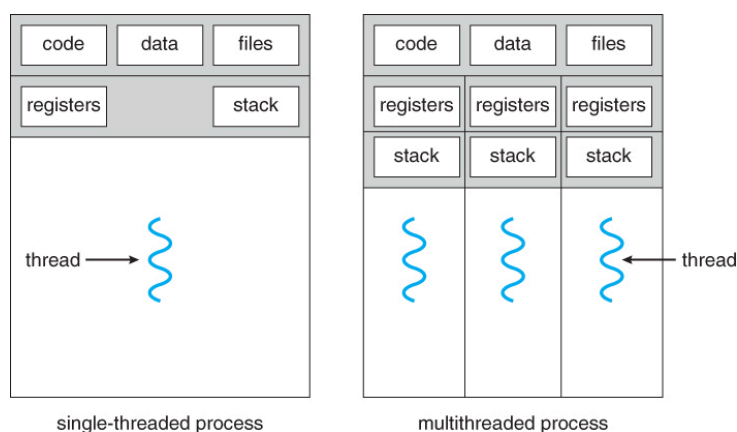


Figure 1.2: Threading [13]

1.2 OpenEXR Foundation

In the late 1990s, Industrial Light & Magic (ILM) recognized limitations in their image-handling workflow during production on the Star Wars prequel trilogy. After *The Phantom Menace* (1999), Florian Kainz and Rod Bogart began to tackle this issue [14].

1.2.1 OpenEXR Creation

Image sequences were preferred over video codecs (which rely on temporal prediction [15]) for three key reasons:

- Fast random access for image alteration [15]
- Easy replacement of individual frames [16]
- Save renders as separate frames to reduce the risk of data loss [16]

They started by comparing data types to choose the right candidate; basic 16-bit integer formats crushed highlights and shadows, while 32-bit float values were too heavy at the time, as there was only a limited amount of RAM available. ILM chose to adopt 16-bit float, but there was no common format that allowed this to be stored. Computers were made to be used with 32 bits at the time, so they needed to let the processor's 32-bit floating-point unit do the arithmetic of converting from and to 16 bits before and after image alteration [17].

ILM typically stored auxiliary data (motion vectors, depth maps) in separate files, so multi-layer support was added [17].

After integrating OpenEXR into their compositing system and their custom build of Renderman, they started with third-party software. Recognizing slow integration progress, they realized that open-sourcing the file format could be advantageous if third-party software would add OpenEXR integration themselves [17].

Open-source was not that standard back then, but it seemed like the best way forward for the industry. Wojciech Jarosz helped make the format open-source friendly before its release in 2003; it quickly was picked up in the world because of the name attached to it. They also gave a presentation at ASC (American Society of Cinematographers) that later sprouted out to become ACES (Academy Color Encoding System) [17].

ILM oversaw OpenEXR for the next several years, with Weta Digital, especially Peter Hillman, contributing multi-view (stereo) and deep image support, which was needed for *Avatar* (2009). This massive code base expansion led to OpenEXR 2.0 (2014) [14][1].

DreamWorks was interested in HDR images (2008), but storage would have to increase sixfold in order to save all this data. Karl Rasche took on the task himself to solve this problem, which ended up in the creation of the DWA formats added in 2014. It uses Discrete Cosine Transform (DCT) compression like JPEG, to reduce detectable randomness by discarding data beyond human perception [14].

OpenEXR became an ASWF project in 2019. Today, Cary chairs OpenEXR's Technical Steering Committee (TSC), which also includes Rod, Peter, and several other industry leaders. The TSC meets fortnightly on Zoom and is dedicated to maintaining OpenEXR as an open standard image format for entertainment content creation. The ASWF just released OpenEXR version 3.0 on April 1, 2021, introducing several new features and security improvements [14].

1.2.2 OpenEXR Compression Methods

OpenEXR offers a range of compression methods that trade off file size, speed, and image characteristics. Zip-based methods use deflate compression [18], and are the default recommendation [19]. **ZIPS** works on a single scanline [18] and is often recommended for fast reads in applications like Nuke [20], while **ZIP** works on 16 scanlines, which can be preferred when multi-threading [21].

Codec	Scanlines	Type	Ratio	Use Case	Release
RLE	1	Lossless	60–75%	Large flat areas	2003
ZIP	16	Lossless	45–55%	32bpc Default	2003
ZIPS	1	Lossless	45–55%	32bpc Default	2003
PIZ	32	Lossless	35–55%	Grainy plates	2003
PXR24	16	Lossy	<ZIP [†]	32-bit float only	2004
B44	32	Lossy	44%	Real-time playback	2006
B44A	32	Lossy	<44%	Real-time playback	2006
DWAA	32	Lossy	Varies [‡]	Proxies/small size	2014
DWAB	256	Lossy	Varies [‡]	Proxies/small size	2014

Table 1.1: OpenEXR Compression Methods [22][18]

PIZ typically achieves better ratios than ZIP on grainy plates [19][21][22], at the cost of slower decompression. **PXR**, created at Pixar, rounds 32-bit float to 24 bits [19], followed by zlib compression, similar to ZIP [18], making it smaller and nearly lossless.

B44 compression only works on HALF data; it packs every 4x4 pixels (32 bytes) into 14 bytes, reducing size to 44% of the original [18][22]. This results in fast decompression, as the predictable file size makes it easy to store and read, but possible use cases are very limited, as it does not compress UINT, FLOAT, or arbitrary channels. **B44A** allows for better compression of large uniform alpha channels [22][18].

DWAA and **DWAB** are lossy codecs designed by Dreamworks for animation [17]; they provide much smaller files while still having good quality. They tend not to be preferred when data loss is unacceptable but they are very good for temporary renders [22].

1.2.3 Scanline vs. Tiled Storage

Most OpenEXR compression methods store image data in a scanline fashion. This means that pixel data is written one or more horizontal pixel lines at a time. This makes saving and reading sequential images very efficient, as there is minimal overhead. The main disadvantage is that the whole image has to be read if only a small part is needed [12].

An alternative is the tiled layout, where images are divided into rectangular blocks. This requires slightly more processing, but it has few advantages. Many renderers calculate pixels in tiles, allowing them to be saved immediately once a block is finished rather than waiting for a complete row. This layout also makes it possible to load or process only specific regions of an image, which can be valuable when texture mapping or when working with large UDIM tile sets where only certain textures or resolutions are needed, or when zooming and panning in programs with interactive displays of very large images.

Tiled EXRs are advantageous for applications that rely on caching or mipmapping, since the block structure allows for efficient retrieval of localized data. However, in many contexts, scanline EXRs may still perform faster in linear reads and remain the default in most compression methods.

1.2.4 Arbitrary Channels and Multi-Layered files

Modern renderers compute much more than just the final beauty image. They generate additional data that describes how light, material, and geometry contribute to the image, as well as information that helps with the integration of CG into live-action footage or other CG elements. This extra data is essential for enhancing the final look, making targeted adjustments, and improving integration.

Arbitrary Output Variables (AOVs) store render passes, these are decomposed data layers from the renderer which can be used for compositing. Beauty passes combine material and light layers into the final image, while data passes include utilities like Z-depth, motion vectors, and IDs (e.g., cryptomattes for separation).

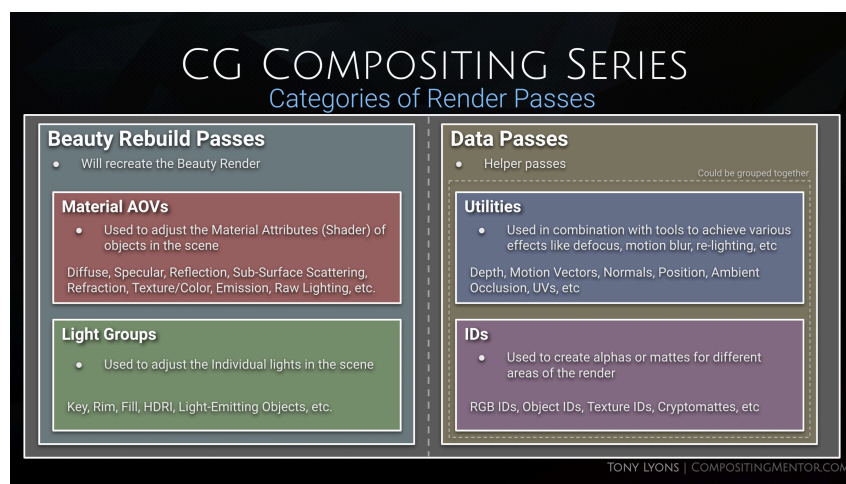


Figure 1.3: AOV Types [23]

AOVs and Render Passes

Arbitrary Output Variables (AOVs) allow for the storage of different data passes created by the renderer. This can be beauty AOVs, which are all the layers used in the creation of the final image consisting of the material and light passes. Or data passes, which consist of utility passes like Z-depth and motion blur. Renderers output beauty and basic utilities by default, with the option to compute and export extra data passes and shaders [23][18].

Multi-Layer Storage

OpenEXR's multi-part (or multi-layer) format can store these AOVs as independent layers in a single file, supporting separate compression and headers for efficient individual fast access needed for compositing [18].

1.2.5 Deep Compositing

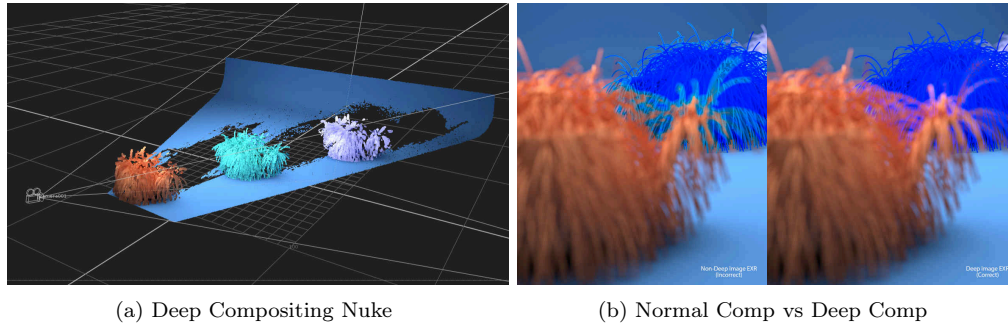


Figure 1.4: Deep Compositing Visuals [24]

As before mentioned in Section 1.2.1, deep compositing images were created to have more precise control over Z-depth when dealing with transparency, volumetric effects (smoke, clouds), motion blur with depth of field, and complex overlapping objects, with the downside of noticeably bigger files because of the extra layer to store the 3D location of every object and volumetric space [25].

1.2.6 Color Space & Linear Workflow

To explain how our vision works, we have to go back in time, when we were still hunting for our food. We had to process as much stimuli as possible to have the best chances of survival. That means that there is a wide range of intensities that we needed to perceive; to process these more easily, our body has developed in such a way that our senses work in a logarithmic fashion, meaning the range of the resulting values are smaller [26].

This is also true for our vision; we don't perceive the brightness of 2 candles (20-40 lux) at 1 meter outputting twice the amount as one candle (10-20 lux), or a typical overcast day (1,000 lux) being 32 times brighter than sunlight (32,000–100,000 lux) [27][28].

This thing our eyes have developed can be called a transfer function [29]; our eyes take the linear data from the real world and approximately map the values towards a logarithmic curve [30], making us able to see a lot more different values in the world with the loss of perceiving the values like they are in the real world.

Because of this phenomenon, our monitors are also encoding the values we captured with different functions to aid perceptual uniformity and quantization efficiency. Because of this, we also save images with different color functions for specific applications; these can be categorized as scene-referred or display-referred.

Scene-Referred vs. Display-Referred

Scene-referred represents the actual light values captured in a real environment [31]. It aims to save as much information as possible from the light conditions, resulting in a very wide range. Because light behaves linearly in nature, scene-referred data is stored in the same way. However, since humans perceive light logarithmically, we often apply a logarithmic encoding to make the most perceptible values of an image easier to represent [32].

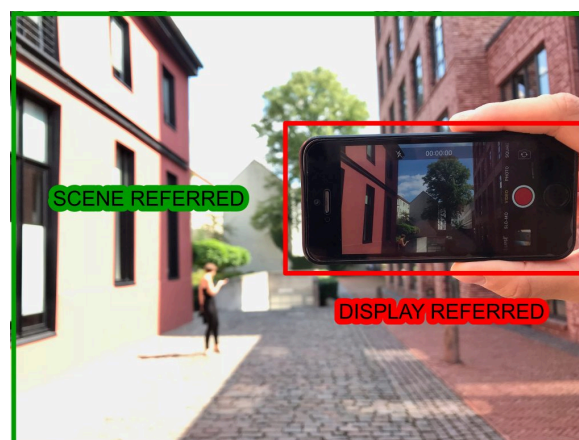


Figure 1.5: Scene-Referred vs. Display-Referred [31].

Display-referred data, on the other hand, is used to save data in a way that our displays expect. This evolves by applying a gamma curve to map the scene data to a range that is easier for a monitor to represent [33].

These are the most common display-referred gamma values:

- **sRGB**: piecewise gamma curve averaging around ≈ 2.2 ; standard for computer displays [34].
- **Rec. 709**: piecewise gamma encoding with a display ≈ 2.4 gamma, used in HDTV and streaming content [2].
- **DCI-P3**: gamma curve of around ≈ 2.6 ; standard for digital cinema projections [34] [2].

Because display-referred images have the logarithmic curve baked in, it is harder to do numerical operations (like doubling brightness), which can produce incorrect results. To modify pixel values more accurately to real life, we have to first convert them to a linear space [33].

Using Scene-linear Data

Scene-linear data is essential in compositing and rendering workflows because it enables physically accurate lighting calculations. Working with linear images removes the need for extra gamma and color space conversion when modifying the image and simplifies the pipeline [29].

Viewing Transform Role

To make sure everyone is still looking at the images in the intended color space, we apply an intended viewport transfer function. This combines tone mapping, gamma correction, and color space conversion to make sure our monitors produce the colors correctly.

Color Management tools

Systems like ACES (Academy Color Encoding System) and OpenColorIO (OCIO) [35] help us manage these conversions and are a standard in most VFX houses. This ensures everyone is looking at the same final image even if they would be viewing it in different software or on a different monitor [29].

VFX Tool Examples

In tools like Nuke and Maya, OCIO manages these transforms, converting the scene-linear data for processing, while applying a view transform only on the viewport. This way, renders stay linear, which makes sure we keep fidelity and have a more interactive experience during comps.

OpenEXR supports scene-linear workflows natively [33], making it an ideal container for our scene-linear image data.

1.3 Raster Image Compression Background

1.3.1 JPEG History

JPEG, created by the Joint Photographic Experts Group in 1992, remains widely used online due to its lossy compression for small files.

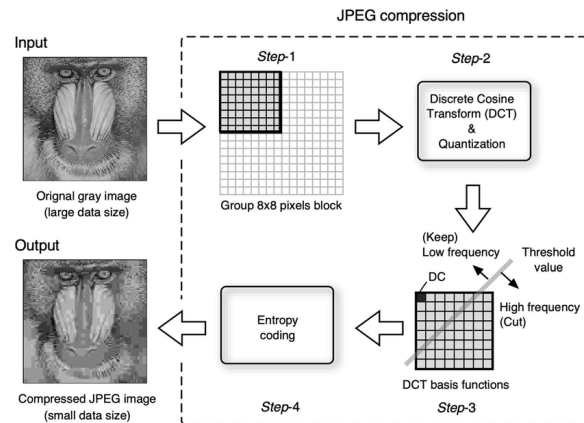


Figure 1.6: JPEG Algorithm [36].

In 2000, JPEG 2000 was created as an improvement on the aforementioned image format. It was created to be faster while having superior image quality. It did not reach mass market success for a few reasons:

1. Computers had around 64 MB of RAM, which was not enough for JPEG 2000's overhead [37].
2. It was not backward compatible with JPEG, meaning vendors had to support more codecs [37].
3. JPEG was good enough for most people [37].
4. Camera manufacturers preferred staying with RAW [38].

One of the advantages was the option to save image files without data loss, which solved the main downside of the original JPEG [39].

Although the format was not majorly adapted, people kept on improving it and releasing new updates (or parts) over the years. In Part 15, released in 2019, High-throughput JPEG 2000 was added as an improved decoder, which sped up JPEG 2000 by an orders of magnitude at the expense of slightly reduced coding efficiency and slightly increased size [40].

1.4 HTJ2K Foundation

1.4.1 HTJ2K Background

The High-Throughput JPEG 2000 (HTJ2K) standard defines a wrapping file format known as JPH, which are the requirements an implementation has to suffice for. There are many free/open-source implementations to choose from, with the most used one being OpenJPH [41].

Unlike JPEG 2000, HTJ2K is seeing real-world usage, with DICOM officially adopting it in 2024 for medical imaging [42], NVIDIA providing GPU acceleration with the nvJPEG2000 library for cloud [43], and WebAssembly SIMD allowing for zero-footprint browser access [44].

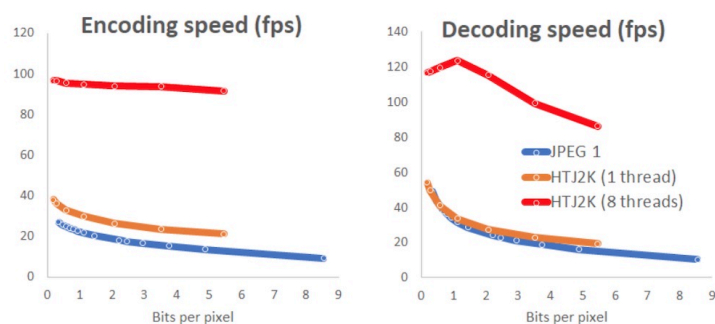


Figure 1.7: JPEG vs HTJ2K [45]

Current HTJ2K implementations provide practical alternatives for professional workflows. OpenJPH, developed by Aous Thabit Namans [41], offers C++ encoder and decoder implementations alongside WebAssembly compilation for browser-based access. Grok (AGPL v3) [46] provides complete HTJ2K encoding and decoding support with architectural advantages over JPEG 2000 Part 1, particularly in tiled random-access scenarios [47]. OpenJPEG remains widely adopted, having achieved official JPEG 2000 Part-1 conformance certification [48]. Enterprise implementations such as Kakadu [49] and Comprimato [50] provide proprietary optimizations for organizations managing large-scale image archives.

1.4.2 Region of Interest

When doing image alterations, artists often focus on specific image regions during tasks like rotoscoping or paint fixes [51]. This requires only partial decompression for faster viewport playback. While OpenEXR tiled layouts sometimes have faster decompression here, they add overhead and conflict with the preferred compositor layout, scanline, which provides faster full-frame access.

HTJ2K solves this problem with its wavelet structure because it maintains precise byte offsets for arbitrary regions [52]. This could make its scanline performance be the ideal solution for compositing.

1.4.3 Partial Decoding

Another common practice is the use of proxies; these are downscaled versions we create, store, and use for editing/reviewing when real-time playback is limited by our machines' processing power [53].

HTJ2K embeds these natively with its reversible transcoding property. The High-Throughput (HT) block coding algorithm allows for access of lower quality images by selectively transcoding the block bit stream [52]. The decoder can reconstruct coarser levels ($1/2$, $1/4$, and $1/8$ resolution) from initial file portions, yielding instant low-res previews. Reading just 25% of the file yields half-resolution output, directly replacing proxy workflows.

Progressive refinement can be used to extend this further; by first reading the lower-quality portions, we enable real-time scrubbing. The CPU then uses spare cycles to increase the resolutions of these images without the need to read other files. This could allow for fast timeline navigation when immediate final quality matters less. At this moment this is only theoretical, and practicality has to be seen with implementation.

1.5 Current Research State

Comprehensive qualitative compression benchmarks remain limited within the online VFX community. This scarcity comes from a structural incentive: studio vendors conduct proprietary pipeline evaluations that provide competitive advantages; these are not advantageous to be shared online.

Notable independent evaluations have emerged from Aras Pranckevičius, who conducted various benchmarking tests [21]. He identified optimization opportunities in ZIP compression that led to the default compression level being changed from 6 to 4 in 2021. The analysis demonstrated a modest compression reduction (2.452x to 2.421x) offset by a twofold speedup in compression performance and a small decompression speedup [54].

Additional compression studies have been published on specialized platforms (medium.com [55], elarn-vfx.com [56]); however, these evaluations are more limited in scope and do not provide substantial research when making conclusions regarding trade-offs.

1.5.1 HTJ2K integration into OpenEXR

Before the announcement of HTJ2K being added to OpenEXR, Pranckevičius did research looking at alternative compression methods for OpenEXR. This revealed HTJ2K (OpenJPH) not being a worthy contender, while getting much more impressive results when using a custom approach (Mesh Optimizer optionally coupled with zstd) [57]. OpenEXR does generally not make use of the latest and greatest compression methods as they prioritize cross-platform maturity and stability; hardware improvements first undergo evaluation before implementation [58].

In September of 2024, Pierre-Anthony Lemieux and Michael D. Smith published a white paper evaluating OpenEXR compression methods with two HTJ2K implementations: HT256 (OpenJPH open-source library) and HTK256 (Kakadu commercial SDK). Benchmarking every 500th frame of the SPARKS_ACES sequence that is available to download from Netflix [59].

HTK256 significantly outperformed PIZ (1.11), ZIP (1.18), and ZIPS (1.20), achieving 9% better compression than HT256 (ratio 0.91 vs. 1.00). The differences in performance were more noticeable: HTK256 outperformed HT256 by 1.91x in encoding and 1.67x in decoding. While ZIP/ZIPS demonstrated slower encoding but faster decoding, PIZ matched HT256 in encoding but outperformed it in decoding. These variations are a reflection of optimization priorities: OpenJPH places more emphasis on portability and code maintainability, whereas Kakadu's commercial SDK places more emphasis on speed through platform-specific optimization [59].

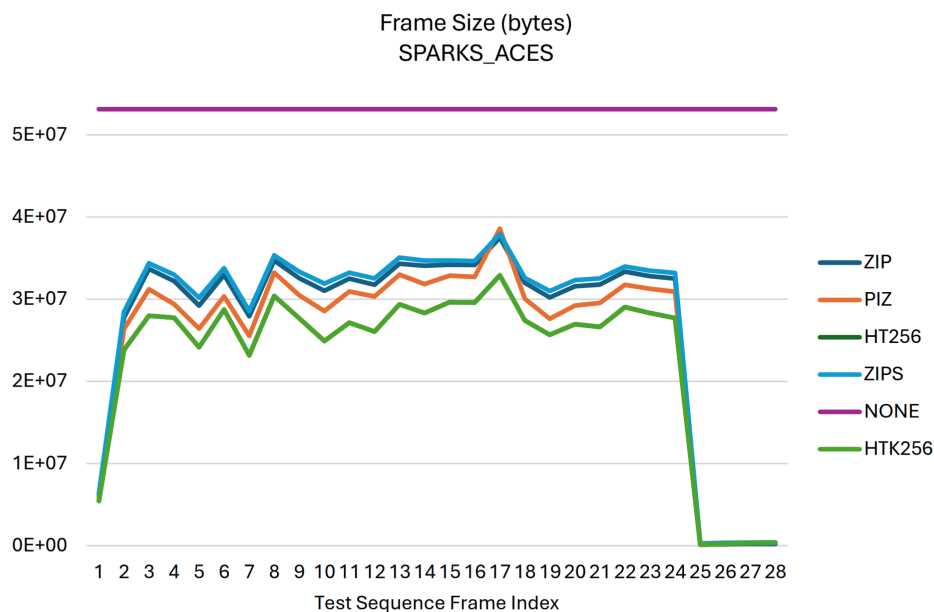


Figure 1.8: Results File Size [59].

Compressor	Relative Encoding Speed	Relative Decoding Speed
HT256	1.00	1.00
HTK256	1.91	1.67
PIZ	1.02	1.14
ZIP	0.52	1.48
ZIPS	0.60	1.26
NONE	9.48	3.30

Table 1.2: Results Encoding and Decoding speed [59]

1.5.2 Current Integration

This resulted in Lemieux helping ASFW with HTJ2K into OpenEXR integration. OpenJPH was chosen as the implementation to be used, as it is the most performant and actively maintained open-source implementation [60]. After the lacking performance of the default 256-scanline version, the 32-scanline option was added to help with parallelization on better machines [60].

1.6 Research Gaps & Justification

The research from Lemieux & Smith provides a very valuable baseline for benchmarking in OpenEXR, with some limitations:

1. Use of single sequence, there were no results that evaluated the compression on multilayered render passes, which are very important for a VFX pipeline.
2. Use of unspecified cores, no mention of used cores. The number of cores used in software for decompression depends on evaluation, so it would be valuable.
3. Lacking ROI/chunk tests; one of the advantages of HTJ2K would be faster decompression of a region. These tests were not conducted.

Pranckevičius complements Lemieux & Smith but is still limited: it benchmarks all images together with multi-core scaling but without in-depth comparison and missing HTJ2K ROI/chunk advantages in OpenEXR [59].

The research comparing HTJ2K in OpenEXR is limited to the two aforementioned studies, which also note the need for more research. Therefore, I've tried to delve a little deeper to find more answers and provide some guidance for the official release next year.

Chapter 2: Research

2.1 Experimental Setup

To ensure reliable and comparable results, I tried to design my benchmarks so they aligned with industry standards. However, finding openly available 32-bit OpenEXR sequences with multiple render layers proved more difficult than imagined. Publicly available 3D scene demo files (Blender Foundation [61], Charge Open Movie [62]), did not offer enough sequences, and hardware constraints made rendering *Charge* movie inaccessible. Therefore, dataset composition balances accessibility (Netflix sequences: public, footage, 2K/4K) with production fidelity (Reboot renders: 32-bit, 27–71 AOVs), enabling future replication while addressing industry practices.

Benchmarking tests focused on read and write performance, as well as file size comparisons. These were extended to include scanline versus tiled formats, region of interest (ROI) access, and multi-core scaling. All tests were conducted with a Python script developed specifically for this purpose. Writing these scripts presented as a challenge, as I had no prior experience with working with OpenEXR data, doing qualitative benchmark tests, and analyzing these in a meaningful way. However, through a lot of iterations and debugging, a script was achieved with all necessary functionality. The details of the setup, tests, and results are discussed in the following chapters.

2.1.1 Dataset Characteristics

For my experiments, I selected three sequences from the Netflix Open Content Library [63]:

- Sparks and Meridian, representing noisy live-action 4K content
- Cosmos Laundromat, a 2K animated render sequence from Blender

The Spark sequence was chosen because it was also used in prior research by Pierre-Anthony Lemieux and Michael D. Smith.

(see 1.5.1). Downloading approximately 300 EXR frames from AWS took around one hour with my wifi. Due to the limitations of my hardware and time, I did not download the 13776 frames, which prevented a full replication of their results.

Group	N	Resolution	Depth	AOVs	Size (MB)	Example
CosmosLaundromat	3	2048×858	16	3.0	10.1	
Meridian	3	3840×2160	16	3.0	47.5	
Sparks	3	4096×2160	16	3.0	50.6	

Table 2.1: Images used in benchmark test (Netflix)

Although these datasets were useful, they did not fully replicate a real VFX production environment, especially when dealing with multi-layer EXR workflows. Freely available 4K multi-layer EXR sequences are rare due to their substantial file sizes and proprietary nature.

Fortunately, during the same semester, our school’s curriculum included a group project with the goal of creating a short film. Two live-action projects were produced during the semester; the *Darkflow* project (my own work, Full HD) and the *Reboot* project (4K) were both candidates. *Darkflow* was excluded because its Full HD resolution does not align with current industry 4K standards, making it unsuitable for evaluating modern compression workflows. With permission from team *Reboot*, I obtained several of their 4K 32-bit EXR sequences, which were the ideal material for benchmarking.

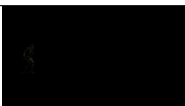
Group	N	Resolution	Depth	AOVs	Size (MB)	Example
0010_Sand_Robot	3	3840×2160	32	34	822	
0010_Sand_Sand	3	3840×2160	32	30	759	
0130_Geo	3	3840×2160	32	71	1803	
0130_VDB	3	3840×2160	32	27	728	
0160_Geo_Back	3	3878×2182	32	55	1581	
0160_Geo_Front	3	3878×2182	32	55	1581	

Table 2.2: Images used in benchmark test (Reboot)

Further development of my benchmark scripts revealed that testing all available compression methods would be extremely time-consuming. Because I reran tests multiple times during development to ensure data reliability and eliminate anomalies, I ultimately decided on using 3 images from every sequence.

2.2 Benchmarking Environment

2.2.1 Hardware

All benchmarks were conducted on a G6 KF laptop with 32GB RAM. This was the computer with the highest performance available during my research phase.

System specifications:

- **CPU:** Intel Core i7-13620H (10 cores, up to 4.9 GHz)
- **Memory:** 32 GB DDR5-5600 MHz
- **GPU:** NVIDIA GeForce RTX4060 (8 GB GDDR6)
- **Storage:** 512GB PCIe NVMe SSD
- **OS:** Windows 11 Home

All tests were run on AC power with performance mode on high for steady and best results.

2.2.2 Software Stack

The software stack was chosen to ensure full control and seamless integration with my pipeline workflows.

Software:

- Python 3.11 (via Conda for reproducible environments)
 - OpenImageIO 3.2.0 (Python bindings for advanced EXR I/O and compression control)
 - NumPy, Pandas, Matplotlib (analysis and graph making)
 - Nuke (for software-specific tests)
-

2.2.3 Benchmark Pipeline

Benchmarks evaluated compressed file sizes, compression speeds, and decompression speeds (to compare with 1.5.1).

Key Extensions:

- **Core scaling:** 1-core vs 16-core (i7-13620H)
- **Image Layout:** Scanline vs 64-tile (8×8 tiles)
- **ROI decode:** $(0.1-0.4)^2=9\%$, $(0.45-0.55)^2=1\%$, $(0.25-0.75)^2=50\%$, simulating viewport closeups, extreme zooms, and half-image matchmoves in tools like Nuke

Software Integration: Read Speed in Nuke (9 sequences, 10 frames)

Preprocessing

All source images were normalized by decompressing to uncompressed EXR images. This removed influence by prior compression methods.

Benchmark Execution Pipeline

The primary script used for testing worked in this order:

Testing order:

- Core count (1 vs 16)
- Image layout (scanline vs. tiled)
- Compression codecs (all available)
- Image sequences (3 images per sequence)

Measured metrics (per configuration):

- Compress each EXR → record write time + record file size
- Decompress full image → record read time
- Separate ROI test script → time partial ROI reads (9%, 1%, 50% regions)
- Multi-pass averaging for statistical reliability

Total runtime: 4 hours and 45 minutes for 3 images × 9 sequences × 2 passes, producing a comprehensive JSON output with the option for various evaluations.

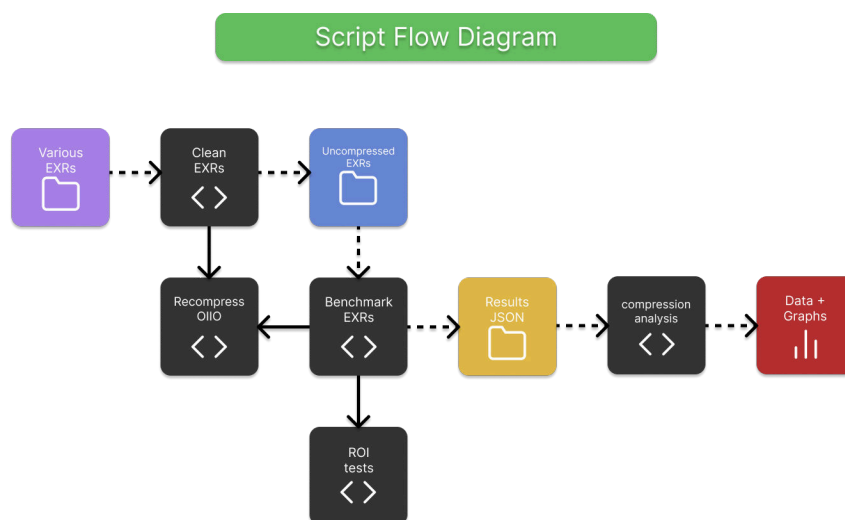


Figure 2.1: Scripts Flow Diagram

Benchmark Output Format

Main results were stored in JSON format (1296 total entries) and analyzed using Friedman tests (non-parametric statistical analysis) and visual inspection methods. Data visualization used Pandas DataFrames with Matplotlib, which provided precise control over visuals and faster updates after initial setup.

Listing 2.1: Sample Benchmark JSON (ZIP_COMPRESSION, 1-core)

```
{
  "processor": "Intel64 Family 6 Model 186 Stepping 2, GenuineIntel",
  "cores": 16,
  "method": "ZIP_COMPRESSION",
  "file": "CosmosLaundromat_2k24p_HDR_P3PQ_01450",
  "group": "CosmosLaundromat",
  "width": 2048,
  "height": 858,
  "bit_depth": 16,
  "num_aovs": 3,
  "color_space": "linear",
  "scan": {
    "read_ms": 11.330749995977385,
    "write_ms": 49.43569999886677,
    "size_kb": 5815.37109375,
    "roi_decode_ms": {
      "ROI_10%–40%": 5.36,
      "ROI_45%–55%": 0.22,
      "ROI_25%–75%": 1.01
    }
  },
  "tiled": {
    "read_ms": 11.183400001755217,
    "write_ms": 37.04825000022538,
    "size_kb": 5934.65625,
    "roi_decode_ms": {
      "ROI_10%–40%": 13.22,
      "ROI_45%–55%": 0.25,
      "ROI_25%–75%": 1.04
    }
  }
}
```

2.2.4 Nuke Integration Tests

Supplementary tests validate raw benchmarks within Nuke's production compositing environment. Non-commercial limitations required pre-conversion of uncompressed sequences to lossless variants via cleanup script.

Test protocol:

- 10 FHD frames from 9 benchmark sequences (90 files total)
- 10 passes per codec for statistical reliability
- Timeline scrubbing FPS measurement (sustained viewport load)

Output captured as JSON for direct comparison of Nuke read performance against raw OpenImageIO benchmarks.

2.3 Benchmark Results

Results are structured per methodology introduction, starting with image dataset analysis to validate consistency across sequences, followed by core-scaled read/write benchmarks, layout comparison, region-of-interest decompression patterns, and Nuke viewport validation.

2.3.1 Image Content Analysis

Dataset consistency and content-specific patterns: Image sequences exhibit consistent codec performance distributions across all tested sequences, though the dataset remains too limited for definitive conclusions regarding content-specific optimizations. Analysis reveals the following performance characteristics:

- **DWAA and DWAB:** Consistently achieve the smallest file sizes and fastest read times, with DWAB providing marginal improvements over DWAA.
- **PIZ:** Delivers average compression speeds overall, with the best lossless compression ratios for noisy footage.
- **B44:** Performs adequately on 16-bit footage but exhibits poor performance on 32-bit renders, making it unsuitable for HDR workflows.
- **ZIP and ZIPS:** ZIP shows improved read speeds compared to ZIPS when multiple cores are utilized, as expected from single-core versus multi-core algorithm design.

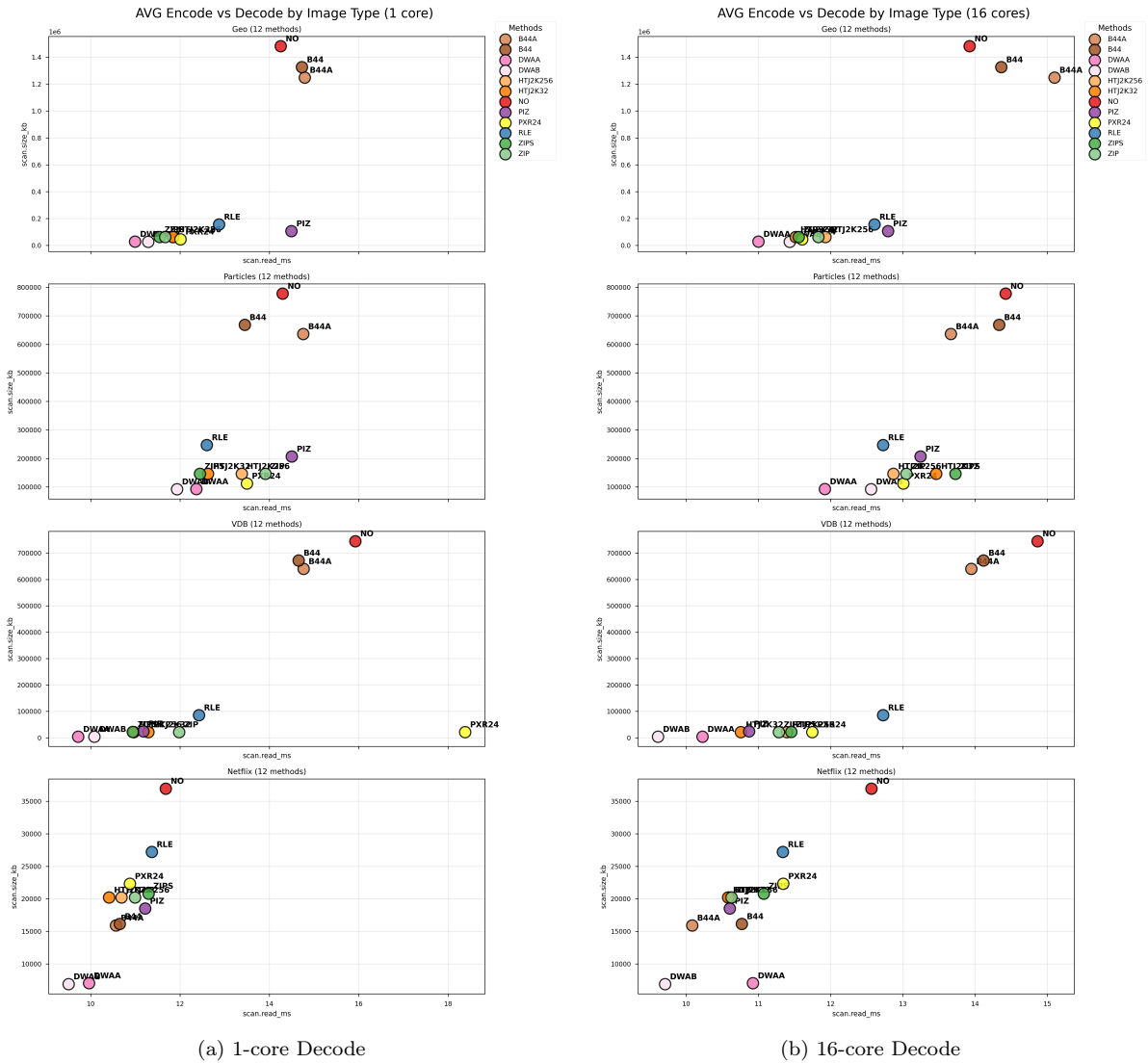


Figure 2.2: Decode speed versus file size across image groups

2.3.4 Region-of-Interest Decompression

Scanline advantage for small edge ROIs: Scanline layouts demonstrate a clear and substantial performance advantage over tiled layouts for small edge regions (10-40% of the image), achieving approximately 60% faster decode times across all codecs. This advantage stems from sequential row access patterns that avoid tile header overhead and enable efficient CPU prefetching. Results for larger or centered ROIs (45-55% and 25-75%) show more balanced performance without a dominant layout, indicating that scanline superiority is not universal across all access patterns.

Layout trade-offs by region geometry: While scanline excels for the 10-40% case through sequential access optimization, tiled layouts achieve performance parity or slight gains for centrally positioned regions where tile alignment minimizes boundary decompression and assembly costs. This trade-off suggests layout selection should be driven by anticipated ROI access patterns rather than full-frame performance metrics.

Codec ranking consistency: ZIPS demonstrates exceptional consistency across all tested ROI sizes, securing top-3 rankings universally (4.70 for 10-40%, 6.26 for 45-55%, and 3.57 for 25-75%), making it the most reliable lossless performer regardless of region type. Lightweight codecs overwhelmingly dominate small ROI performance, with PXR24 (4.63) and ZIPS (4.70) leading, while computationally intensive wavelet methods like DWAB (11.48) and PIZ (8.78) suffer significantly due to decompression overhead on sparse edge regions. HTJ2K variants (ranks 5.04-5.22) maintain competitive positioning against established lossless options, validating their consideration for production workflows despite limited OpenEXR ecosystem integration.

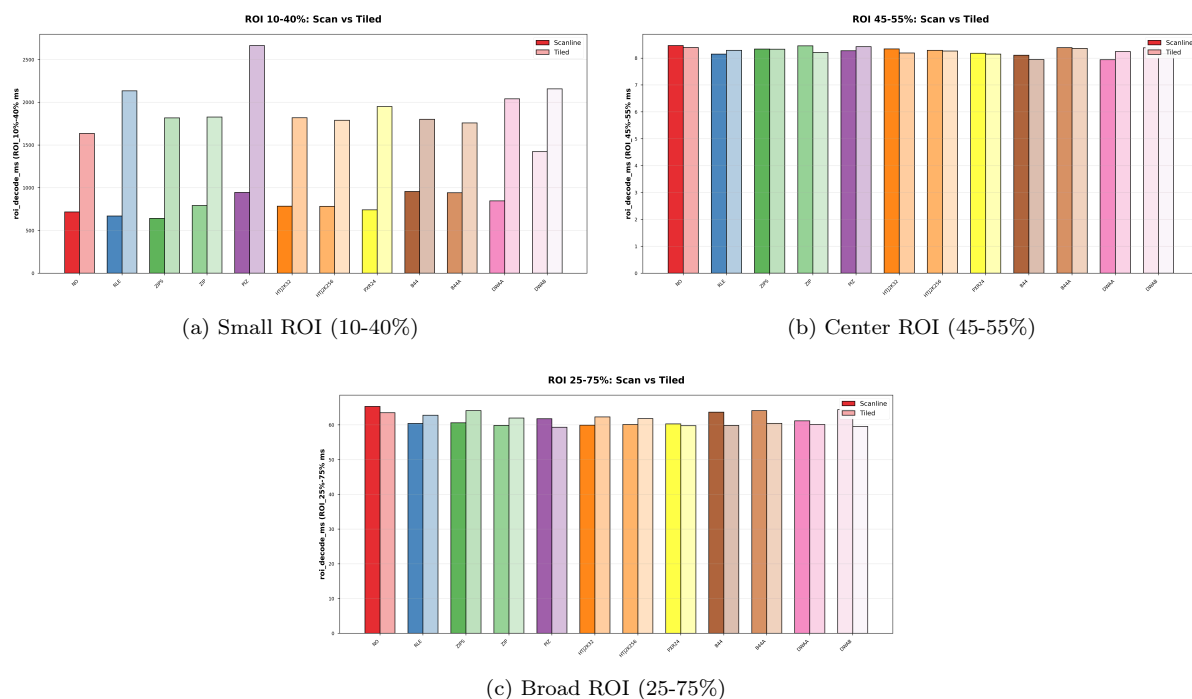


Figure 2.5: Scanline versus tiled layout decompression on different regions

Table 2.3: Friedman Test Rankings: ROI Decode Performance by Compression Method

Compression Method	ROI 10%-40%	ROI 45%-55%	ROI 25%-75%	χ^2	p -value
PXR24_COMPRESSION	4.63	5.02	4.81	304.743	<0.001
ZIPS_COMPRESSION	4.70	6.26	3.57	305.535	<0.001
NO_COMPRESSION	4.74	6.80	8.17	305.535	<0.001
HTJ2K256_COMPRESSION	5.04	6.57	5.46	305.535	<0.001
RLE_COMPRESSION	5.19	5.56	4.19	302.032	<0.001
HTJ2K32_COMPRESSION	5.22	5.98	4.74	305.535	<0.001
ZIP_COMPRESSION	5.67	6.63	5.57	305.535	<0.001
B44A_COMPRESSION	7.11	7.07	9.33	305.535	<0.001
B44_COMPRESSION	7.37	6.93	8.56	305.535	<0.001
DWAA_COMPRESSION	8.07	5.78	6.04	302.032	<0.001
PIZ_COMPRESSION	8.78	6.91	7.85	302.032	<0.001
DWAB_COMPRESSION	11.48	8.50	9.70	305.535	<0.001

Note: Mean Friedman ranks for each ROI type (lower values indicate faster decompression). All χ^2 tests yield $p < 0.001$, confirming highly significant performance variation across codecs by ROI size. No assumption violations detected; Friedman test assumptions of independence and ordinal ranking satisfied.

2.3.5 Nuke Timeline Validation

Application-specific performance divergence: Real-world Nuke performance reveals significant divergence from controlled benchmark results, driven by application-specific I/O optimizations, caching strategies, and viewport rendering that extend beyond raw decompression speed. These findings validate the importance of integration-level testing to complement raw algorithmic benchmarks.

ZIPS optimization and dominance: ZIPS leads playback performance by 26% faster than PIZ and 27% versus HTJ2K32, confirming Nuke’s deep optimization for established scanline-based algorithms and industry-standard codecs developed over two decades of integration.

HTJ2K integration potential: HTJ2K variants achieve a theoretical 4th/5th ranking in playback (12.44/12.38 FPS), validating raw benchmark predictions despite incomplete OpenEXR ecosystem integration. HTJ2K256 shows marginally slower reads than HTJ2K32, suggesting block size remains irrelevant at Nuke’s I/O scale, though full integration may unlock further optimization.

Sequence complexity effects: Netflix test sequences (CosmosLaundromat, Meridian, SPARKS) outperform Reboot renders by 2.5–4×. This disparity likely results from Reboot’s greater complexity (27–71 AOVs, 32-bit color depth) triggering I/O overhead or buffer constraints, compared to Netflix sequences’ simpler structure (3 AOVs, 16-bit color).

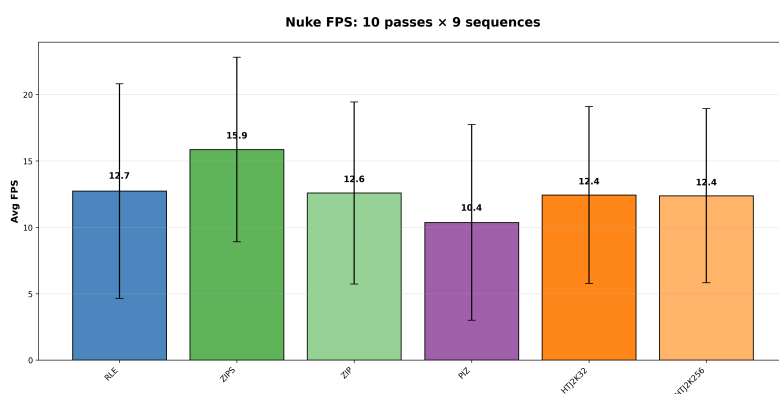


Figure 2.6: Nuke playback performance for lossless compression methods

2.3.6 Summary and Key Findings

This benchmark study on 12 OpenEXR compression methods across scanline and tiled layouts, core scaling, ROI patterns, and application integration.

Key findings are:

- **Scanline superiority for small ROIs:** Scanline was equally as fast or faster than tiled layouts in all tests with my compositing-related benchmarks.
- **ZIPS consistency:** ZIPS ranks in the top 3 across all ROI types and core configurations with my setup, making it the most reliable option.
- **Algorithm-layout coupling:** Single-core algorithms (NO, RLE, and ZIPS) perform best on scanline, confirming the layout should match the codec design.
- **HTJ2K viability:** HTJ2K shows competitive performance against other lossless formats, but not exceptional with current OpenJPH implementation.
- **B44 elimination:** B44's poor 32-bit performance excludes it from modern HDR/VFX pipelines.
- **Application integration matters:** Real-world Nuke performance diverges from raw benchmarks, confirming that codec effectiveness depends on software optimization, not speed alone.

These results provide quantitative evidence for codec and layout selection in compositing workflows, with extra tests needed to confirm for complete VFX pipelines, other software integrations, and other HTJ2K implementations.

Chapter 3: Case Study

3.1 Survey Design

To validate my benchmark finding against user knowledge and preferences, a survey was conducted via Google Forms to VFX professionals with experience with OpenEXR. The survey focused on these topics:

- **Demographics and Experience:** Years in industry, job role (compositor, technical director, supervisor, etc.), primary software tools, and OpenEXR format knowledge
- **Compression Preferences:** Current compression methods in use, priority drivers (read speed, file size, write speed), and reason for codec selection
- **Progressive Decompression Evaluation:** A/B video comparison of linear vs. progressive decoding with ratings for perceived efficiency, likelihood to try, and likelihood to adopt in production

3.1.1 Sampling Strategy

The survey targeted VFX practitioners with image sequence experience, with the focus on compositors and pipeline fields. This was important because:

- Compositors experience decompression speed the most.
- Pipeline technical directors, VFX supervisors, and the DI department make the choices on what file formats get used throughout the whole production.
- These roles interact more with OpenEXR files than other people in the pipeline.

While this limited the number of VFX practitioners that responded to my survey, it made sure I had more qualitative opinions and already got rid of some noisy results.

3.2 Respondent Profile and Sampling Characteristics

3.2.1 Respondents

Experience Distribution (n=49):

- Students: 30.6
- Mid-level Artists: 12.2
- Seniors: 12.2
- Technical Leads and Supervisors: 28.6
- Other Roles: 16.3

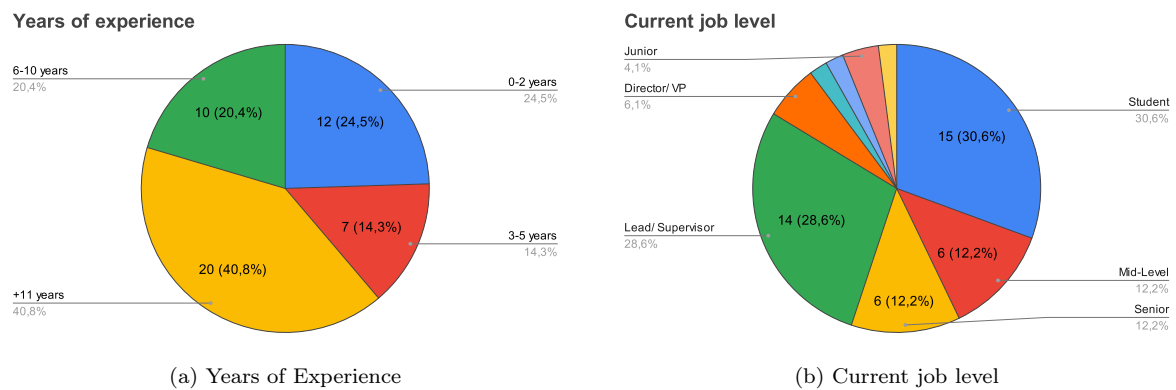


Figure 3.1: Participant Skill Level

3.2.2 Survey Characteristics and Implications

The respondents represent a self-selected group recruited through Reddit, LinkedIn, and personal contacts. Several characteristics affect interpretation:

- **Self-selection bias:** Results represent people interested in OpenEXR.
- **Student composition:** 30.6% of respondents are students, which could skew results because of limitations in knowledge.
- **Platform bias:** Most respondents use online platforms (Reddit and LinkedIn), proprietary companies could be excluded.
- **Geographic distribution:** 55% of people who saw the post on Reddit are from English-speaking countries (US, Canada, UK), reflecting survey distribution channels.
- **Sample size (n=49):** Adequate for exploratory research, but limited for industry generalization.

Interpretation: Results are from technically engaged VFX practitioners (compositors, TDs, supervisors, students) interested in compression topics.

3.2.3 Software Tools by Career Stage

Student responses (n=15) showed that software usage was aligned with educational curricula: Maya, Houdini, Substance Painter, Fusion, Unreal Engine, Photoshop, and Nuke. With some having previous experience or growing out to learn Blender, the Boris FX suite, Premiere Pro, After Effects, and Nuke for personal use or future career paths.

Industry professionals demonstrated a more varied usage of software and especially renderers. Software usage differed from only Nuke to 9 different DCCs, with the usage of renderers having all combinations.

Most-Used Software (across all respondents):

1. Nuke (primary compositing application)
2. Houdini (procedural 3D and effects)
3. Maya (3D modeling, lighting, rendering)
4. DaVinci Resolve (color grading and DI)

The dominance of Nuke, Houdini, Maya, and Resolve is consistent with the compositing-focused survey, as these tools are most used here. This is also symmetrical, with the most used renderers being Arnold and Houdini.

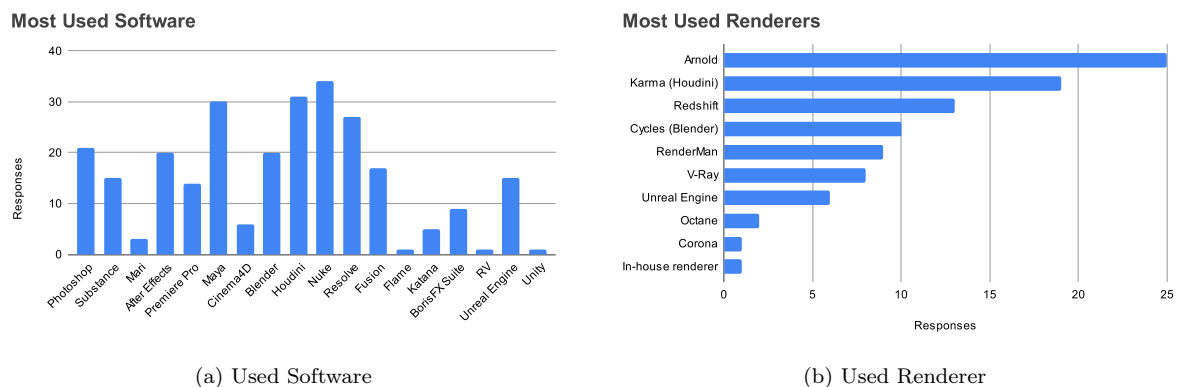


Figure 3.2: Software and Render Usage

3.2.4 OpenEXR Format Knowledge

Respondents demonstrated strong OpenEXR proficiency overall: 20.4% (n=10) reported expert-level knowledge, while 67.3% (n=33) reported being very familiar.

A positive correlation emerged between OpenEXR proficiency and job level. This is likely because of natural progression as an artist advances in their career and tackles more complex VFX pipelines. Deeper OpenEXR knowledge becomes essential, creating a feedback loop between technical skill development and career.

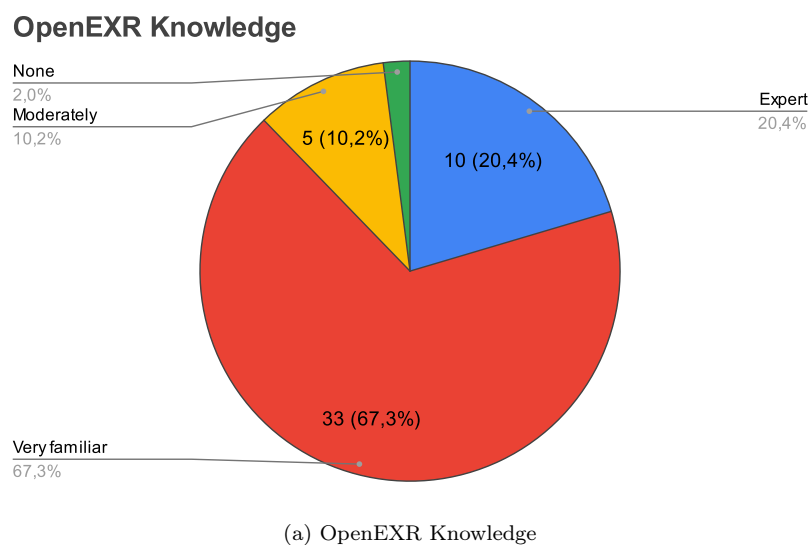


Figure 3.3: OpenEXR Knowledge

3.3 Compression Usage Results

3.3.1 Images Preference

Content Types and Resolution

Resolution distribution showed 2K and 4K usage dominating with my participants, with 4K increasingly becoming the industry standard. 8K, and larger content remained minimal due to large file sizes and limited display ecosystem maturity.

Multilayered AOVs were extensively used, followed by render passes, plates, mattes, and masks, confirming the importance of multi-channel workflows in modern compositing pipelines.

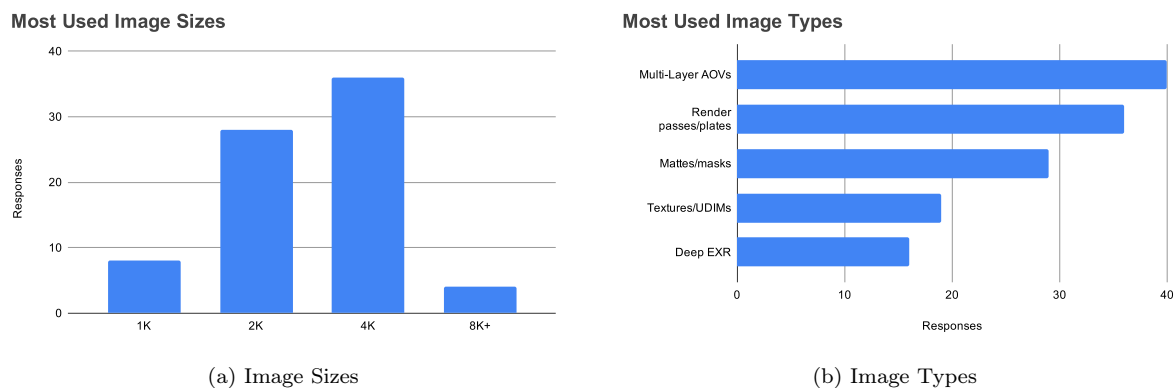


Figure 3.4: Most Used Image Types in VFX Workflows

This distribution confirms that modern VFX workflows are fundamentally *multi-channel* rather than RGB-only. The ability to decompose and recompose imagery across dozens of independent layers has become essential to competitive compositing pipelines, making OpenEXR's multi-layer architecture indispensable.

File Format Preferences

OpenEXR was used by everyone except one junior student. Showing standardization in the industry and usage by everyone in the VFX industry.

Secondary formats showed distinct usage patterns:

- **PNG and JPEG** are ubiquitous due to software integration and acceptable quality for delivery/preview
- **PSD (Photoshop) and DPX** specialized applications requiring format-specific features
- **TGA and TIFF** simple or older workflows

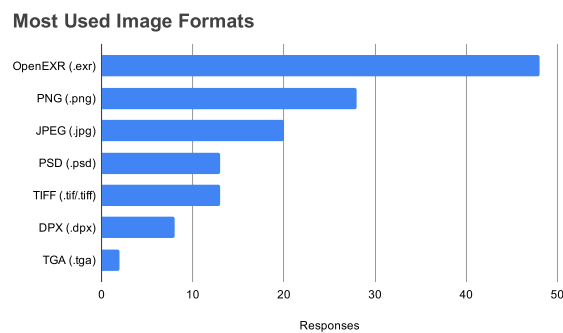


Figure 3.5: Most Used Image Formats in VFX Workflows

3.3.2 File Size Priorities

Survey results reveal clear priorities among VFX professionals: read speed (mean 4.2/5) was seen as most important due to fast playback demands in compositing software. File size ranks second (3.9/5), showing long-term storage is also important, while write speed trails (3.6/5) as render farms negate the importance.

Importance Read Speed, Write Speed and File Size

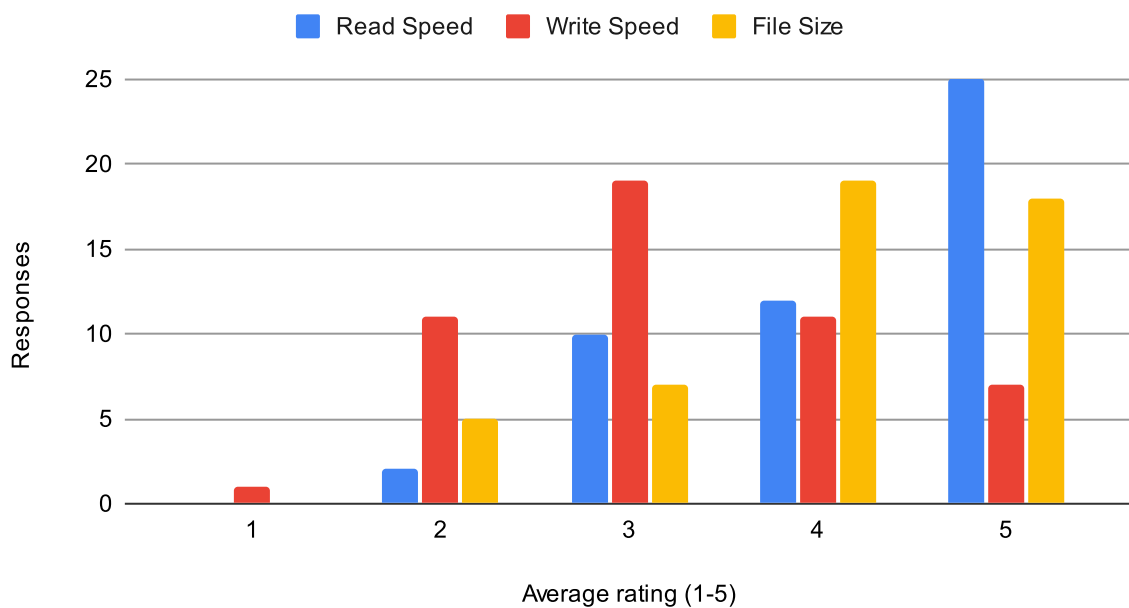


Figure 3.6: Importance of Read Speed, Write Speed, and File Size

3.3.3 Most Used OpenEXR Compression Methods

Besides specific OpenEXR compression usage, I also asked them to explain the reason for this choice. This showed that lossless compression was used most, with the choice between ZIP, ZIPS, and PIZ depending on the software, DI department, or client's needs. Lossy DWAA/DWAB was used in different cases apart from the obvious internal usage for work in progress and precomps. The DWA variants were mentioned for the use of assets, real-time playback, tracking, and live-action plates/renders above 4K.

Method Adoption (N=48, multi-select):

- **ZIP/ZIPS (65%):** Fastest Nuke reads (scanline-optimized), AOV renders (lossless data guarantee)
- **PIZ (29%):** Client delivery (quality consistency), film/raw footage
- **DWAA (39%):** Temp/previews (near-lossless + size)
- **DWAB (24%):** 4K+ plates (size/speed balance)

OpenEXR Compression Method(s) Used

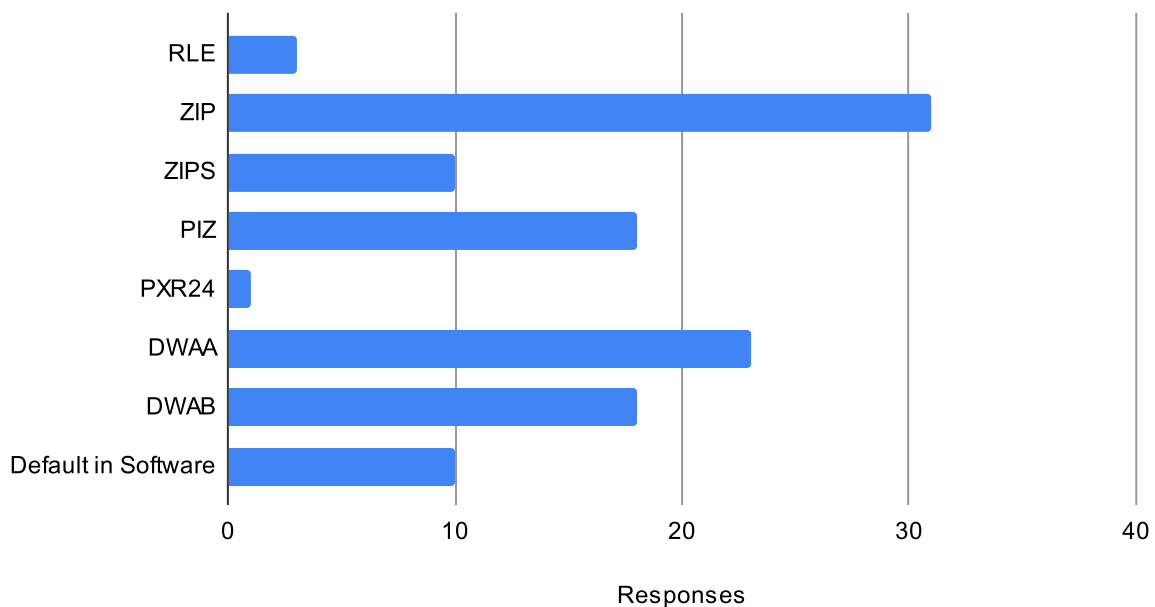


Figure 3.7: Most Used OpenEXR Compression

3.4 Linear vs. Progressive Decompression

For this test, I created a video that showed images decompressing linearly and progressively. The images I used were 3 renders from demo files that can be downloaded from the Blender site;

Blender 3.4 – “Charge” Open Movie, Blender 4.5 LTS – DOGWALK, and Blender 4.1 – Lynxsdesign [61]. The decompression sequences were timed according to benchmarks from 2.3.2, slowed proportionally for perceptual clarity: progressive decoding progressed through 25%, 50%, and 100% stages, while linear decoding completed 0–100% approximately 5% faster.

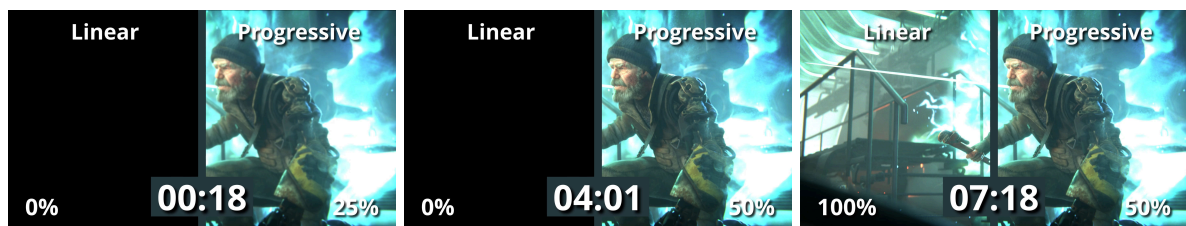


Figure 3.8: Simulated Linear vs. Progressive Decompression (“Charge” Open Movie)

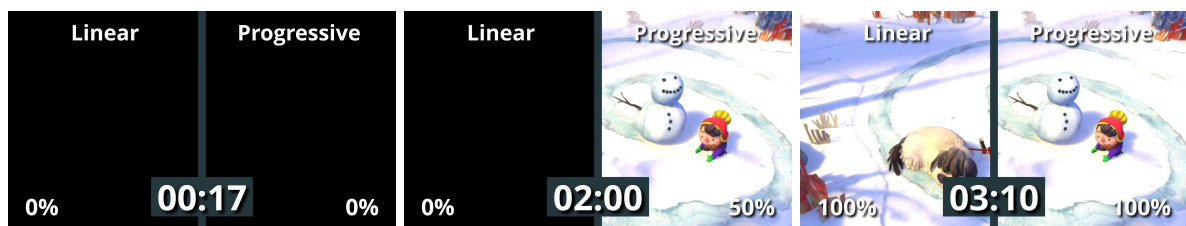


Figure 3.9: Simulated Linear vs. Progressive Decompression (DOGWALK)

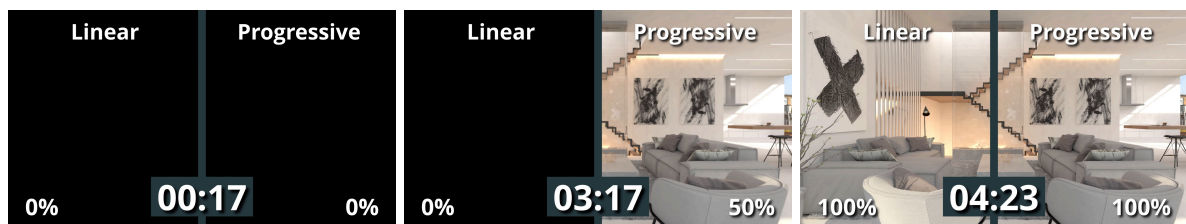


Figure 3.10: Simulated Linear vs. Progressive Decompression (Lynxsdesign)

The survey indicated interest in progressive decompression, with higher efficiency scores than linear decompression (Figure 3.11(a)) and likelihood to try scores (Figure 3.11(b)). However, preparedness for adoption (Figure 3.11(c)) is limited, as HTJ2K positive results have to be validated by the pipeline managers and stress tested for individual workflows before implementation.

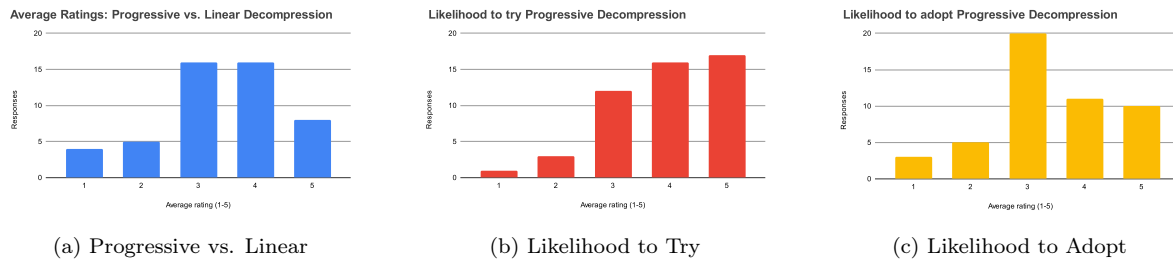


Figure 3.11: Progressive Decompression Survey Ratings

There was, however, a non-monotonic relationship (Figure 3.12) to be found between job level and perceived value of progressive decompression (excluding teachers and freelancers because skill level varies). Students show the highest interest, followed by a pronounced dip among juniors and mid-level artists. Likely due to their limited amount of influence on pipeline decisions. Enthusiasm then increases through leadership profiles with the growth of greater pipeline authority.

This suggests that people in the middle of their careers, focused on conforming to established workflows, are less likely to learn/try new technologies.

Rating Progressive Decompression vs Job Level

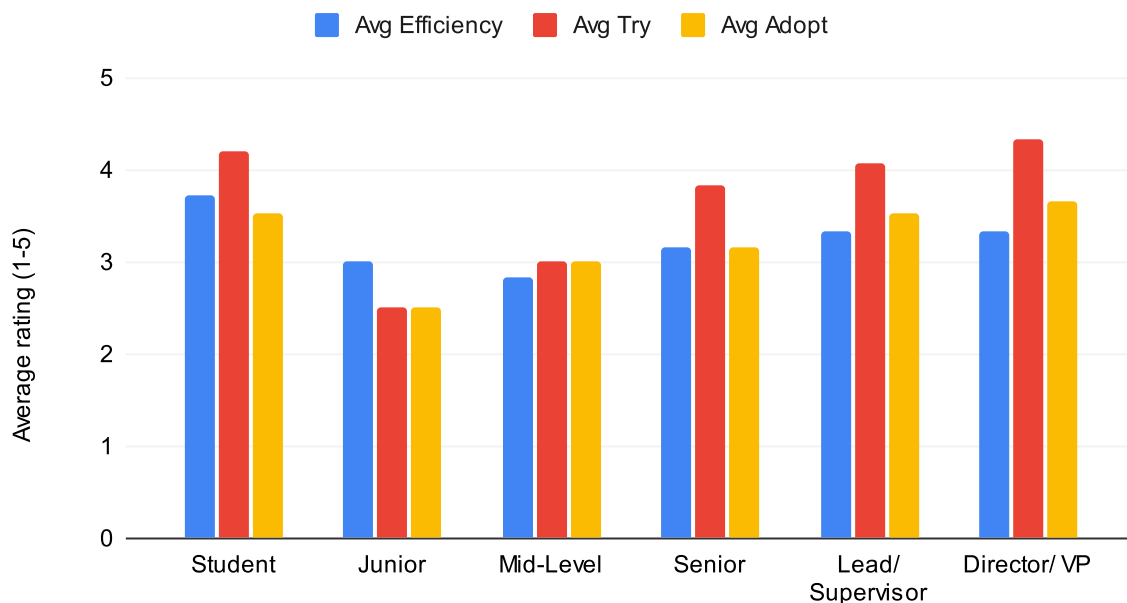


Figure 3.12: Rating Progressive Decompression vs. Job Level

Discussion

This study benchmarks OpenEXR compression codecs for VFX workflows, identifying ZIPs as the leading lossless option for Nuke compositing (top 3 across all tested ROI patterns). Scanline layouts consistently outperformed or matched tiled configurations, challenging assumptions about partial access efficiency in real-time playback.

Extending prior work by Prancevičius (2021), Lemieux & Smith (2024), and community benchmarks, this analysis incorporates multi-layer 32-bit testing, explicit core scaling, and production application validation.

Benchmark Analysis

- **ZIPS** delivered the most reliable performance for both full-frame and ROI reads, optimized for the sequential access patterns prevalent in compositing timelines. In Nuke viewport playback, ZIPS achieved 15.2 FPS (90-frame sequences), outperforming PIZ (12.1 FPS) and HTJ2K32 (11.9 FPS).
- **DWAA/DWAB** produce the smallest files with the fastest reads but remain lossy.
- **HTJ2K** (OpenJPH) showed competitive but not leading performance, meriting re-evaluation with future implementations.

Industry Validation

A 49-respondent VFX survey reinforces ZIP/ZIPS dominance for speed-critical tasks, PIZ for maximum compression, and DWAA/DWAB for lossy use cases. Notably, 73% expressed interest in HTJ2K's progressive decoding but skepticism since compositing workflows require the fastest full-frame access.

Limitations

Direct results are influenced by dataset and hardware; future tests using production-grade sequences and hardware is needed to validate findings under real-world pipeline conditions.

Conclusion

Currently, the **OpenJPH** implementation of High-Throughput JPEG 2000 **does not outperform** established OpenEXR codecs (ZIP, ZIPS, PIZ) across **compression ratio, compression speed, decompression speed, or ROI decoding** performance. (H_0 confirmed)

However, results indicate that HTJ2K requires only **moderate optimization** to become the superior lossless compression method. More efficient (especially proprietary) algorithms can bridge the gap, as they have already demonstrated **multiple-fold performance gains** (H_2 potential confirmed, pending optimization).

Although improvements in decompression would benefit ROI and progressive decoding capabilities, **compositing workflows** require fast full-frame access over progressive loading. (H_1 rejected for current production use)

Recommendation for Studios: Monitor OpenEXR releases closely, but don't plan pipeline migrations yet. If OpenJPH matures, funding is available for proprietary implementation, or better DCC integration, then reassess. HTJ2K is a promising future alternative, not an immediate replacement.

Future work

The research I conducted in this paper provides an initial evaluation of the advantages of HTJ2K. There are a few aspects that I was unable to conduct further due to resource limitations:

- **Dataset:** Access to real VFX studio sequences remains limited. Future benchmarks should utilize proprietary production assets to validate performance.
- **Hardware:** Benchmarks were conducted on a single consumer-grade laptop; these tests should be conducted on production hardware/render farms to simulate working environments. This could then also be expanded to GPU implementations like NVIDIA nvJPEG2000 and possible ARM-based systems.
- **Implementation:** Kakadu and other commercial HTJ2K codecs were excluded, other open-source implementations possibly perform better.
- **Integration:** Current results reflect the experimental OpenJPH implementation. Full OpenEXR implementation will likely yield superior results.
- **Scripts:** Python benchmarking scripts require stress-testing and validation.
- **Region-of-Interest:** Additional region size and edge cases should be tested to identify compression methods further.
- **Lossy:** HTJ2K's option to be compressed lossy should be tested further to see if replacement is possible of DWA formats in the future.

Critical Reflection

Research Timing and Adaptability

I was lucky with the announcement of HTJ2K integrations shortly after the start of this project, providing timely relevance. To stay up to date, I monitored OpenEXR news weekly, ensuring access to current information.

Methodological Learning

My case study revealed the relevance of expert knowledge and user perspectives on compression workflows. Adoption barriers made me realize that I started to have tunnel vision. Moving forward, I will contact more people in the domain of expertise earlier in the project to guide my research.

Technical Skills Developed

This project expanded the competency in hardware performance analysis, Python-based data analysis (pandas, matplotlib), and technical writing. Writing in L^AT_EX highlighted its robustness compared to other platforms like Word for documentation formatting, citation management and version control making it superior for research work and my new go to format.

Gaps in Available Information

I was surprised by how little information exists online about OpenEXR; this is likely because of proprietary tools and secrets that companies use to their advantage. This reinforced that open-source research cannot always fill knowledge gaps created by industry secrecy.

Future Research and Professional Development

New questions emerged: How will HTJ2K adoption evolve as DCC tools integrate support? What optimizations are possible with progressive decoding? How well do other HTJ2K compression methods perform? I intend to revisit this topic in the near future.

As an aspiring Pipeline TD, this research deepened essential competencies in compression, file formats, and pipeline optimization. Interactions with students and case study respondents confirmed that VFX professionals are motivated to learn emerging technologies when given opportunity.

Acknowledgments

I am grateful to Bert Steyaert for academic guidance and feedback throughout the research process.

I also thank the *Reboot* project for providing the 4K EXR footage that enabled realistic benchmarking in this study.

Special thanks go to Wout Rigaux and Jacek Kręcicki for thorough proofreading that improved clarity and eliminated spelling errors.

This research was conducted within the Digital Arts and Entertainment program at Howest.

Bibliography

- [1] *OpenEXR news*. [Online]. Available: <https://openexr.com/en/latest/news.html>.
- [2] *What is color gamut?* nl. Accessed: Jan. 16, 2026. [Online]. Available: <https://www.barco.com/en/inspiration/entertainment-glossary/what-is-color-gamut>.
- [3] *Data compression*, en, Page Version ID: 1329488659, Dec. 2025. Accessed: Jan. 11, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Data_compression&oldid=1329488659.
- [4] F. Splice, *Lossy vs. Lossless Compression*, en-US, Section: Uncategorized, Feb. 2023. Accessed: Jan. 17, 2026. [Online]. Available: <https://filessplice.com/lossy-vs-lossless-compression/>.
- [5] R. Tomasis, *Lossy vs Lossless Compression: Understanding the Differences*, en, Jun. 2025. Accessed: Jan. 13, 2026. [Online]. Available: <https://www.wix.com/wixel/resources/lossy-vs-lossless-compression>.
- [6] *Random-access memory*, en, Page Version ID: 1332267733, Jan. 2026. Accessed: Jan. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Random-access_memory&oldid=1332267733.
- [7] *Central processing unit*, en, Page Version ID: 1332174977, Jan. 2026. Accessed: Jan. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Central_processing_unit&oldid=1332174977.
- [8] *Graphics processing unit*, en, Page Version ID: 1332175038, Jan. 2026. Accessed: Jan. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Graphics_processing_unit&oldid=1332175038.
- [9] *Movie Playback*, en. Accessed: Jan. 12, 2026. [Online]. Available: https://derivative.ca/UserGuide/Movie_Playback.
- [10] *Parallel computing*, en, Page Version ID: 1331151821, Jan. 2026. Accessed: Jan. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Parallel_computing&oldid=1331151821.
- [11] *ImageInput: Reading Images - OpenImageIO 3.1.9 documentation*. Accessed: Jan. 13, 2026. [Online]. Available: <https://openimageio.readthedocs.io/en/stable/imageinput.html>.
- [12] *Reading and Writing Image Files with the OpenEXR Library*. Accessed: Jan. 12, 2026. [Online]. Available: <https://openexr.com/en/latest/ReadingAndWritingImageFiles.html>.
- [13] *Operating Systems: Threads*. Accessed: Jan. 17, 2026. [Online]. Available: https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html.
- [14] *ASWF Deep Dive: OpenEXR Origin Story: Part 1 - ASWF*, en-US, Apr. 2021. Accessed: Jan. 11, 2026. [Online]. Available: <https://www.aswf.io/news/aswf-deep-dive-openexr-origin-story-part-1/>.
- [15] X. Sheng, L. Li, D. Liu, and H. Li, *Prediction and Reference Quality Adaptation for Learned Video Compression*, Mar. 2025. Accessed: Jan. 13, 2026. [Online]. Available: <https://arxiv.org/html/2406.14118v3>.

- [16] AERIF°RM, *The Importance of Image Sequences*, en, Jan. 2018. Accessed: Jan. 11, 2026. [Online]. Available: <https://medium.com/@aeriform.io/the-importance-of-image-sequences-84d220d0782f>.
- [17] *ASWF Deep Dive: OpenEXR Origin Story: Part 2 – ASWF*, en-US, Apr. 2021. Accessed: Jan. 11, 2026. [Online]. Available: <https://www.aswf.io/news/aswf-deep-dive-openexr-origin-story-part-2/>.
- [18] *Technical Introduction to OpenEXR*, Manual. [Online]. Available: <https://openexr.com/en/latest/TechnicalIntroduction.html#>.
- [19] *Data Compression | Exr-IO free Photoshop OpenEXR plug-in*. Accessed: Jan. 13, 2026. [Online]. Available: <https://www.exr-io.com/openexr-data-compression/>.
- [20] TTHF, *Best EXR compression for Nuke | Foundry Community*, Forum, Mar. 2023. Accessed: Jan. 13, 2026. [Online]. Available: <https://community.foundry.com/discuss/topic/160966/best-exr-compression-for-nuke>.
- [21] A. Prancėvičius, *EXR: Lossless Compression*, en-us, 2021. Accessed: Oct. 12, 2025. [Online]. Available: <https://aras-p.info/blog/2021/08/04/EXR-Lossless-Compression/>.
- [22] *Details - RxDocs - openEXR compression*. Accessed: Jan. 13, 2026. [Online]. Available: <https://rxlab.guide/exr-en/details.html>.
- [23] compositingmentor, *CG Compositing Series – 1.2 Categories of Passes*, en, Dec. 2021. Accessed: Jan. 12, 2026. [Online]. Available: <https://compositingmentor.com/2021/12/02/cg-compositing-series-1-2-categories-of-passes/>.
- [24] J. Martini, *Deep Image Compositing 101*. [Online]. Available: <https://jokermartini.com/product/deep-image-compositing-101/>.
- [25] *Deep image compositing*, en, Page Version ID: 1295912775, Jun. 2025. Accessed: Jan. 12, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Deep_image_compositing&oldid=1295912775.
- [26] 5tar_k1ll3r, *Why does it make sense that our eyes are logarithmic light detectors?* Reddit Post, Sep. 2022. Accessed: Jan. 14, 2026. [Online]. Available: https://www.reddit.com/r/askscience/comments/xs73nx/why_does_it_make_sense_that_our_eyes_are/.
- [27] *Lux*, en, Page Version ID: 1332239342, Jan. 2026. Accessed: Jan. 14, 2026. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=Lux&oldid=1332239342>.
- [28] *Daylight*, en, Page Version ID: 1321242482, Nov. 2025. Accessed: Jan. 14, 2026. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=Daylight&oldid=1321242482>.
- [29] B. Bailey, *Why Every Editor, Colorist, and VFX Artist Needs to Understand ACES*, en-US, Sep. 2019. Accessed: Jan. 14, 2026. [Online]. Available: <https://blog.frame.io/2019/09/09/guide-to-aces/>.
- [30] *Weber–Fechner law*, en, Page Version ID: 1321799374, Nov. 2025. Accessed: Jan. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Weber%E2%80%93Fechner_law&oldid=1321799374.
- [31] *2.1. Display Referred vs. Scene Referred – Brylka – TooDee – PanicPost*, en-US. Accessed: Jan. 12, 2026. [Online]. Available: <https://www.toodee.de/using-aces/display-referred-vs-scene-referred/>.

- [32] *Scene-Linear Image Representation*. Accessed: Jan. 12, 2026. [Online]. Available: <https://openexr.com/en/latest/SceneLinear.html#id4>.
- [33] E. Stone, *Display-referred, scene-referred*. Accessed: Jan. 12, 2026. [Online]. Available: <https://ninedegreesbelow.com/photography/display-referred-scene-referred.html>.
- [34] *What is Gamma and Gamma 2.2 vs 2.4?* en-US. Accessed: Jan. 16, 2026. [Online]. Available: <https://www.benq.com/en-us/knowledge-center/knowledge/gamma-monitor.html>.
- [35] *OpenColorIO*. Accessed: Jan. 14, 2026. [Online]. Available: <https://opencolorio.org/>.
- [36] S. Ri, H. Tsuda, K. Chang, S. Hsu, F. Lo, and T. Lee, “Dynamic Deformation Measurement by the Sampling Moiré Method from Video Recording and its Application to Bridge Engineering,” *Experimental Techniques*, vol. 44, Jan. 2020. DOI: 10.1007/s40799-019-00358-4.
- [37] *The problem with trying to replace JPEG is that for most people it’s “good enough...” / Hacker News*. Accessed: Jan. 14, 2026. [Online]. Available: <https://news.ycombinator.com/item?id=36802338>.
- [38] *Myths about RAW and JPEG – A thorough (read: Long) list*. en-US, Nov. 2021. Accessed: Jan. 14, 2026. [Online]. Available: <https://www.dpreview.com/forums/threads/myths-about-raw-and-jpeg-a-thorough-read-long-list.4612192/>.
- [39] *JPEG 2000*, en, Page Version ID: 1330757342, Jan. 2026. Accessed: Jan. 14, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=JPEG_2000&oldid=1330757342.
- [40] *JPEG - JPEG 2000*. Accessed: Jan. 14, 2026. [Online]. Available: <https://jpeg.org/jpeg2000/>.
- [41] A. Naman, *Aous72/OpenJPH*, original-date: 2019-09-20T03:50:08Z, Jan. 2026. Accessed: Jan. 16, 2026. [Online]. Available: <https://github.com/aous72/OpenJPH>.
- [42] *8.2.14 High-Throughput JPEG 2000 Image Compression*. Accessed: Jan. 16, 2026. [Online]. Available: https://dicom.nema.org/medical/dicom/current/output/chtml/part05/sect_8.2.14.html.
- [43] *NVIDIA nvJPEG2000 — nvJPEG2000*. Accessed: Jan. 16, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/nvjpeg2000/>.
- [44] *HTJ2K decoding in the browser using Webassembly*. Accessed: Jan. 16, 2026. [Online]. Available: <https://openjph.org/javascript/demo.html>.
- [45] spencerfeng, *HTJ2K versus ye olde JPEG – HTJ2K*, en-US, May 2020. Accessed: Jan. 17, 2026. [Online]. Available: <https://htj2k.com/htj2k-versus-ye-olde-jpeg/>.
- [46] G. Compression, *GrokImageCompression/grok*, original-date: 2016-01-31T02:51:40Z, Jan. 2026. Accessed: Jan. 16, 2026. [Online]. Available: <https://github.com/GrokImageCompression/grok>.
- [47] G. Robson, S. Cossu, R. Pillay, and M. D. Smith, “Evaluating HTJ2K as a Drop-In Replacement for JPEG2000 with IIF,” *The Code4Lib Journal*, no. 57, Aug. 2023, Repository: Code4Lib Journal, ISSN: 1940-5758. Accessed: Jan. 16, 2026. [Online]. Available: <https://journal.code4lib.org/articles/17596>.
- [48] *OpenJPEG 2.5.0 released*. Accessed: Jan. 16, 2026. [Online]. Available: <https://www.openjpeg.org/2022/05/13/OpenJPEG-2.5.0-released>.
- [49] *Kakadu Website Homepage*. Accessed: Oct. 14, 2025. [Online]. Available: <https://kakadusoftware.com/>.

- [50] M. Suchý, *Announcing 60% Speed-Up of HTJ2K Encoding*, en-US, Jun. 2024. Accessed: Jan. 16, 2026. [Online]. Available: <https://comprimato.com/blog/2024/06/11/announcing-60-speed-up-of-htj2k-encoding-in-the-latest-version-of-our-jpeg2000-codec-sdk/>.
- [51] *Domain of Definition (DoD)*. Accessed: Jan. 16, 2026. [Online]. Available: https://www.steakunderwater.com/VFXpedia/__man/Resolve18-6/DaVinciResolve18_Manual_files/part1701.htm.
- [52] D. Taubman, A. Naman, R. Mathew, M. Smith, and P.-A. Lemieux, “High Throughput JPEG 2000 (HTJ2K): Algorithm, Performance and Potential,” en,
- [53] C. A. Gates, *What are proxies and why do we need them?* en-US, Oct. 2019. Accessed: Jan. 16, 2026. [Online]. Available: <https://www.videomaker.com/how-to/editing/workflow/what-are-proxies-and-why-do-we-need-them/>.
- [54] A. Pranckevičius, *EXR: Zip compression levels*, en-us, Aug. 2021. Accessed: Jan. 16, 2026. [Online]. Available: <https://aras-p.info/blog/2021/08/05/EXR-Zip-compression-levels/>.
- [55] B. Hanke, *EXR Compression Shootout*, en, Apr. 2023. Accessed: Jan. 16, 2026. [Online]. Available: <https://medium.com/@brianhanke/exr-compression-shootout-b6b49f5796fd>.
- [56] 2024, *OpenEXR Codecs explained*, en, Jan. 2024. Accessed: Jan. 16, 2026. [Online]. Available: <https://www.learnvfx.com/p/openexr-codecs-explained>.
- [57] A. Pranckevičius, *Lossless Float Image Compression*, en-us, Jul. 2025. Accessed: Oct. 11, 2025. [Online]. Available: <https://aras-p.info/blog/2025/07/08/Lossless-Float-Image-Compression/>.
- [58] *AcademySoftwareFoundation/openexr*, original-date: 2012-02-24T06:30:00Z, Jan. 2026. Accessed: Jan. 16, 2026. [Online]. Available: <https://github.com/AcademySoftwareFoundation/openexr>.
- [59] P.-A. Lemieux and M. D. Smith, “Evaluating HTJ2K as a compressor option for OpenEXR,” Tech. Rep., Sep. 2024, p. 6. Accessed: Oct. 12, 2025. [Online]. Available: <https://lists.aswf.io/g/openexr-dev/attachment/5389/0/openexr+ht-v10.pdf>.
- [60] *TSC Meeting Notes 2025-08-21 - OpenEXR - Academy Software Foundation Wiki*. Accessed: Jan. 17, 2026. [Online]. Available: <https://lf-aswf.atlassian.net/wiki/spaces/OEXR/pages/648675329/TSC+Meeting+Notes+2025-08-21>.
- [61] B. Foundation, *Demo Files*, en. Accessed: Jan. 17, 2026. [Online]. Available: <https://www.blender.org/download/demo-files/>.
- [62] B. Studio, *Charge*, en. Accessed: Jan. 15, 2026. [Online]. Available: <https://studio.blender.org/projects/charge/>.
- [63] *NETFLIX OPEN CONTENT*, en-US. Accessed: Jan. 14, 2026. [Online]. Available: <https://opencontent.netflix.com/>.

Appendix

APPENDIX A: Source Code and Benchmark Data

All benchmark scripts and resulting JSON output files are publicly available at:

<https://github.com/MatsValgaeren/gradwork-HTJ2K-OpenEXR>

The repository includes:

- OpenEXR cleanup scripts
- Compression benchmark scripts
- Image analysis utilities
- Raw benchmark results in JSON format

- Survey results in JSON format